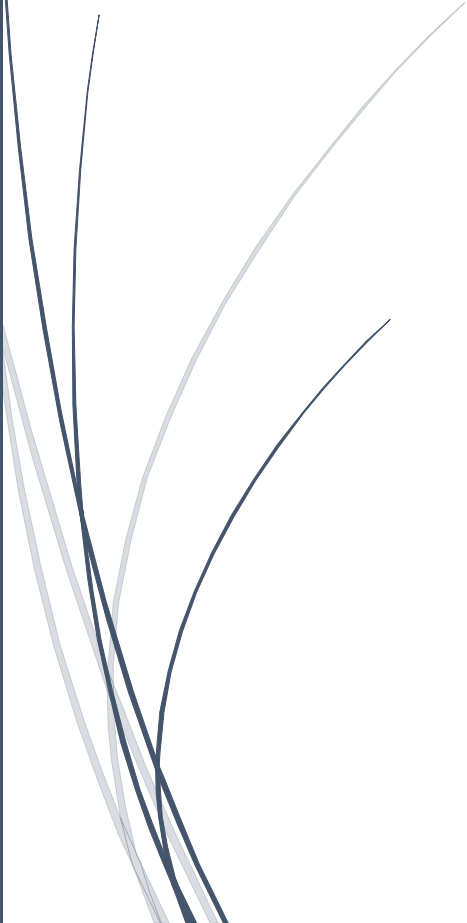


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the date.

18-5-2026

Documentación

CONTENERIZACIÓN DEL STACK COMPLETO CON DOCKER Y DOCKER COMPOSE

Several thin, curved lines in shades of blue and grey that sweep upwards from the bottom left corner of the page.

**NOMBRE DEL ESTUDIANTE: DANIEL ALTAMIRANO
INSTITUTO SUPERIOR TECNOLÓGICO CEMLAD**

Índice

1. Introducción
2. Objetivos
3. Marco Teórico
4. Desarrollo del Proyecto
5. Dockerización del Backend Django
6. Dockerización del Frontend Angular
7. Orquestación con Docker Compose
8. Variables de Entorno y Seguridad
9. Verificación y Funcionamiento
10. Implementación Docker
11. Conclusiones
12. Recomendaciones
13. Referencias Bibliográficas

1. Introducción

Uno de los problemas más comunes en el desarrollo de software es la frase “funciona en mi máquina”. Esto ocurre cuando una aplicación funciona correctamente en el computador del desarrollador, pero presenta errores al ejecutarse en otros entornos.

Para resolver este problema se utilizan contenedores. Los contenedores permiten empaquetar una aplicación junto con todas sus dependencias y configuraciones necesarias para ejecutarse de manera idéntica en cualquier máquina.

En esta actividad se realizó la contenerización completa del sistema desarrollado durante el semestre, integrando:

- Backend Django REST Framework.
- Frontend Angular.
- Base de datos PostgreSQL.

Todo el stack fue orquestado utilizando Docker Compose, permitiendo levantar toda la aplicación mediante un único comando.

Esta práctica permitió comprender conceptos importantes relacionados con DevOps, despliegue de aplicaciones y automatización de entornos.

2. Objetivos

2.1 Objetivo General

Contenerizar el stack completo de la aplicación utilizando Docker y Docker Compose para lograr un entorno reproducible, portable y fácil de ejecutar.

2.2 Objetivos Específicos

- Crear un Dockerfile para el backend Django.
- Crear un Dockerfile para el frontend Angular.
- Configurar PostgreSQL mediante Docker Compose.
- Implementar variables de entorno seguras.
- Automatizar el inicio de los servicios.
- Comprender la integración completa de aplicaciones contenerizadas.

3. Marco Teórico

3.1 Docker

Docker es una plataforma que permite crear, ejecutar y distribuir aplicaciones mediante contenedores.

Un contenedor es un entorno aislado que incluye todo lo necesario para ejecutar una aplicación:

- Código fuente.
- Dependencias.
- Librerías.
- Variables de entorno.
- Configuraciones.

Docker facilita el despliegue de aplicaciones y garantiza que funcionen de la misma manera en cualquier entorno.

3.2 Diferencia entre Contenedor y Máquina Virtual

Las máquinas virtuales simulan sistemas operativos completos, mientras que los contenedores comparten el kernel del sistema anfitrión.

Máquinas virtuales

- Mayor consumo de recursos.
- Más pesadas.
- Inicio más lento.

Contenedores

- Más ligeros.
- Inicio rápido.
- Menor consumo de memoria.
- Mejor rendimiento.

3.3 Dockerfile

Un Dockerfile es un archivo que contiene instrucciones para construir una imagen Docker.

Dentro de este archivo se especifica:

- Imagen base.
- Instalación de dependencias.
- Copia de archivos.
- Puertos.
- Comandos de inicio.

3.4 Docker Compose

Docker Compose permite ejecutar múltiples contenedores utilizando un único archivo de configuración llamado docker-compose.yml.

Esto facilita la orquestación de aplicaciones compuestas por varios servicios.

En este proyecto Docker Compose administra:

- Backend Django.
- Frontend Angular.
- PostgreSQL.

3.5 Multi-Stage Build

La construcción multi-stage permite utilizar múltiples etapas dentro de un Dockerfile.

Esto ayuda a:

- Reducir el tamaño final de la imagen.
- Eliminar archivos innecesarios.
- Mejorar la seguridad.
- Optimizar el rendimiento.

3.6 Variables de Entorno

Las variables de entorno permiten almacenar información sensible fuera del código fuente.

Ejemplos:

- SECRET_KEY
- DATABASE_URL
- POSTGRES_PASSWORD

Esto evita exponer credenciales dentro del repositorio.

4. Desarrollo del Proyecto

4.1 Descripción General

El proyecto consistió en contenerizar completamente la aplicación desarrollada durante el semestre.

El stack final quedó compuesto por:

Servicio	Tecnología
Frontend	Angular
Backend	Django REST Framework
Base de datos	PostgreSQL
Orquestación	Docker Compose

Todos los servicios fueron configurados para ejecutarse automáticamente utilizando Docker Compose.

4.2 Arquitectura del Stack

El sistema funciona mediante tres servicios principales:

1. Frontend Angular.
2. Backend Django.
3. PostgreSQL.

El frontend consume la API del backend, mientras que el backend se conecta a PostgreSQL para almacenar la información.

5. Dockerización del Backend Django

5.1 Imagen Base Utilizada

Para el backend se utilizó:

`python:3.12-slim`

Esta imagen fue seleccionada porque:

- Es oficial.
- Tiene tamaño reducido.
- Incluye Python optimizado para producción.

5.2 Construcción Multi-Stage

El Dockerfile del backend fue construido utilizando múltiples etapas.

Etapas 1

Instalación de dependencias.

Etapas 2

Construcción final ligera únicamente con los archivos necesarios.

Esto reduce considerablemente el tamaño de la imagen final.

5.3 Variables de Entorno

El backend utiliza variables de entorno para:

- SECRET_KEY
- DATABASE_URL
- DEBUG
- ALLOWED_HOSTS

Las credenciales no fueron escritas directamente dentro del código.

5.4 Migraciones Automáticas

Se implementó un archivo `entrypoint.sh` encargado de:

- Esperar PostgreSQL.
- Ejecutar migraciones.
- Iniciar el servidor Django.

Esto automatiza completamente el proceso de inicio.

5.5 Dockerfile Backend

```
FROM python:3.12-slim AS deps
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
FROM python:3.12-slim AS runtime
```

```
WORKDIR /app
```

```
COPY --from=deps /usr/local/lib/python3.12 /usr/local/lib/python3.12
```

```
COPY . .
```

```
RUN chmod +x entrypoint.sh
```

```
EXPOSE 8000
```

```
ENTRYPOINT ["/entrypoint.sh"]
```

5.6 entrypoint.sh

```
python manage.py migrate
```

```
python manage.py runserver 0.0.0.0:8000
```

6. Dockerización del Frontend Angular

6.1 Imagen Base Utilizada

Para Angular se utilizó:

```
node:20-alpine
```

para la compilación y:

```
nginx:alpine
```

para servir la aplicación.

6.2 Construcción Multi-Stage

Etapas Build

- Instalación de dependencias.

- Compilación Angular.

Etapas Serve

- Copia únicamente de archivos compilados.
- Servidor Nginx.

La imagen final no contiene Node.js ni archivos TypeScript.

6.3 Dockerfile Frontend

FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build --configuration production

FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html

COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

6.4 nginx.conf

```
server {  
    listen 80;  
    location / {  
        root /usr/share/nginx/html;  
        index index.html;  
        try_files $uri $uri/ /index.html;  
    }  
}
```

La configuración try_files permite que Angular Router funcione correctamente.

7. Orquestación con Docker Compose

7.1 Servicios Implementados

El archivo docker-compose.yml contiene tres servicios:

- db
- backend
- frontend

7.2 Servicio PostgreSQL

Utiliza:

postgres:16-alpine

Se implementó:

- volumen persistente,
- variables de entorno,
- healthcheck.

7.3 Servicio Backend

El backend:

- depende de PostgreSQL,
- utiliza variables .env,
- expone puerto 8000.

7.4 Servicio Frontend

El frontend:

- depende del backend,
- expone puerto 80.

7.5 docker-compose.yml

version: '3.9'

services:

db:

image: postgres:16-alpine

environment:

POSTGRES_DB: app_db

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

volumes:

- postgres_data:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 10s

timeout: 5s

retries: 5

backend:

build: .

depends_on:

db:

condition: service_healthy

env_file:

- .env

ports:

- "8000:8000"

frontend:

build: ./frontend

depends_on:

- backend

ports:

- "80:80"

volumes:

postgres_data:

8. Variables de Entorno y Seguridad

8.1 Archivo .env

El archivo .env contiene:

SECRET_KEY=your_secret_key

DEBUG=True

ALLOWED_HOSTS=localhost,127.0.0.1

DATABASE_URL=postgresql://postgres:postgres@db:5432/app_db

POSTGRES_DB=app_db

POSTGRES_USER=postgres

POSTGRES_PASSWORD=postgres

8.2 Archivo .env.example

SECRET_KEY=

DEBUG=

ALLOWED_HOSTS=

DATABASE_URL=

POSTGRES_DB=

POSTGRES_USER=

POSTGRES_PASSWORD=

8.3 .gitignore

.env

__pycache__/

node_modules/

dist/

*.pyc

Estas configuraciones ayudan a proteger información sensible.

9. Verificación y Funcionamiento

9.1 Ejecución Completa

El stack completo se ejecuta utilizando:

`docker compose up --build`

9.2 Resultado Esperado

Al finalizar:

- PostgreSQL inicia correctamente.
- Django ejecuta migraciones.
- Angular se compila.
- Nginx sirve el frontend.

9.3 URLs Finales

Frontend

`http://localhost`

Backend

<http://localhost:8000>

Swagger

<http://localhost:8000/api/docs/>

10. Implementación Docker

Paso 1 – Instalación de Docker Desktop

Se descargó e instaló Docker Desktop desde el sitio oficial de Docker.

Posteriormente se verificó la instalación utilizando el siguiente comando:

```
docker --version
```

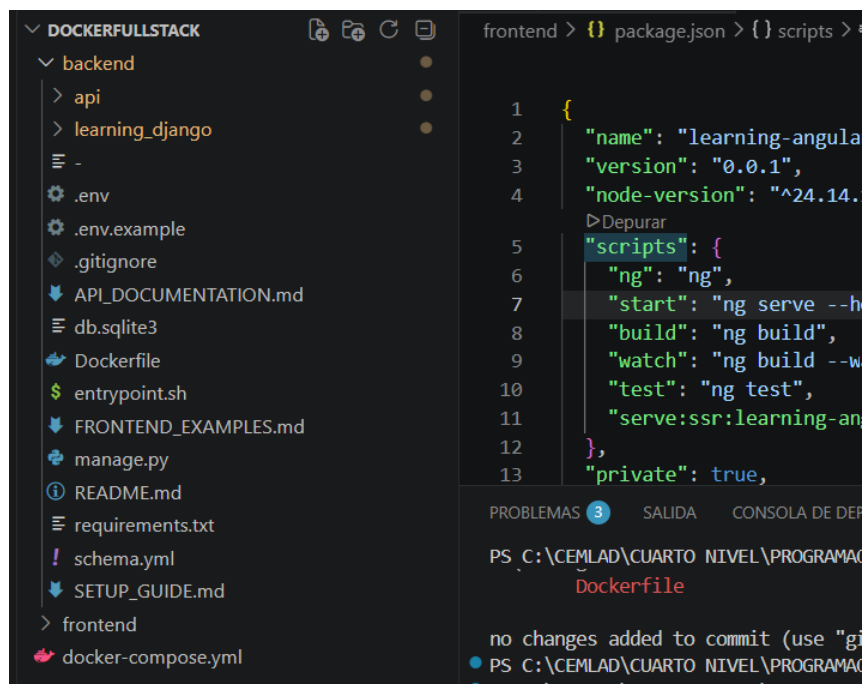
Paso 2 – Clonación de Repositorios

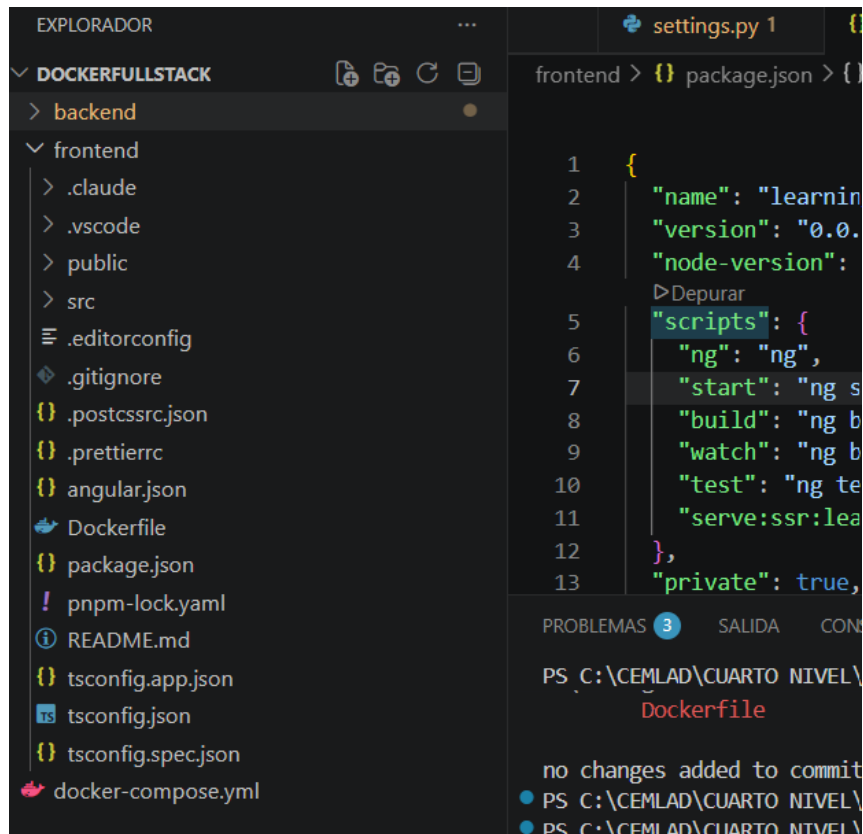
Se clonaron los dos proyectos desarrollados durante el semestre:

- Backend Django REST Framework.
- Frontend Angular.

Ambos proyectos fueron organizados dentro de una carpeta principal llamada:

DockerFullStack





The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure for 'DOCKERFULLSTACK'. The 'backend' folder is expanded, showing a 'frontend' subfolder. Inside 'frontend', various files are listed, including '.editorconfig', '.gitignore', '.postcssrc.json', '.prettierrc', 'angular.json', 'Dockerfile', 'package.json', 'pnpm-lock.yaml', 'README.md', 'tsconfig.app.json', 'tsconfig.json', 'tsconfig.spec.json', and 'docker-compose.yml'. The 'package.json' file is selected and its content is displayed in the main editor. The file contains a 'scripts' section with commands for 'ng', 'start', 'build', 'watch', 'test', and 'serve:ssr:lea'. The 'private' field is set to true.

```
1 {
2   "name": "learnin
3   "version": "0.0.
4   "node-version":
5   "scripts": {
6     "ng": "ng",
7     "start": "ng s
8     "build": "ng b
9     "watch": "ng b
10    "test": "ng te
11    "serve:ssr:lea
12  },
13  "private": true,
```

Paso 3 – Configuración del Backend Django

Dentro del proyecto backend se creó el archivo Dockerfile utilizando Python 3.12 Slim como imagen base.

También se configuró:

- directorio de trabajo,
- instalación automática de dependencias,
- variables de entorno,
- y exposición del puerto 8000.

Paso 4 – Creación del Archivo entrypoint.sh

Se creó el archivo entrypoint.sh para automatizar el inicio del backend Django.

Este archivo ejecuta automáticamente:

- migraciones,
- conexión con PostgreSQL,
- e inicio del servidor Django.

Paso 5 – Configuración de Variables de Entorno

Se creó el archivo `.env` para almacenar configuraciones importantes como:

- SECRET_KEY
- DEBUG
- ALLOWED_HOSTS
- credenciales PostgreSQL

Esto permitió separar información sensible del código fuente.

Paso 6 – Configuración PostgreSQL

El sistema originalmente utilizaba MySQL, sin embargo, durante el proceso de dockerización se configuró PostgreSQL para trabajar correctamente con Docker Compose.

También se actualizó el archivo `settings.py` para conectar Django con PostgreSQL.

Paso 7 – Corrección de Dependencias

Durante la construcción del contenedor backend se identificó un problema relacionado con la librería:

`mysqlclient`

La solución consistió en eliminar dicha dependencia y utilizar:

`psycopg2-binary`

como conector PostgreSQL para Django.

Paso 8 – Configuración del Frontend Angular

En el frontend Angular se creó un Dockerfile utilizando Node.js Alpine.

También se configuró:

- PNPM como gestor de paquetes,
- compilación Angular SSR,
- y exposición del puerto 4200.

Paso 9 – Configuración Docker Compose

Se creó el archivo:

`docker-compose.yml`

para integrar:

- PostgreSQL,

- Backend Django,
- Frontend Angular.

Docker Compose permitió iniciar todos los servicios mediante un único comando:

```
docker compose up --build
```

Paso 10 – Construcción y Ejecución de Contenedores

Se ejecutó Docker Compose para construir automáticamente las imágenes y levantar todos los contenedores del sistema.

Durante este proceso se realizaron múltiples correcciones relacionadas con:

- variables de entorno,
- configuración PostgreSQL,
- dependencias Python,
- Angular SSR,
- exposición de puertos,
- y configuración de Docker.

Paso 11 – Verificación de Contenedores

Se verificó el estado de los contenedores utilizando:

```
docker compose ps
```

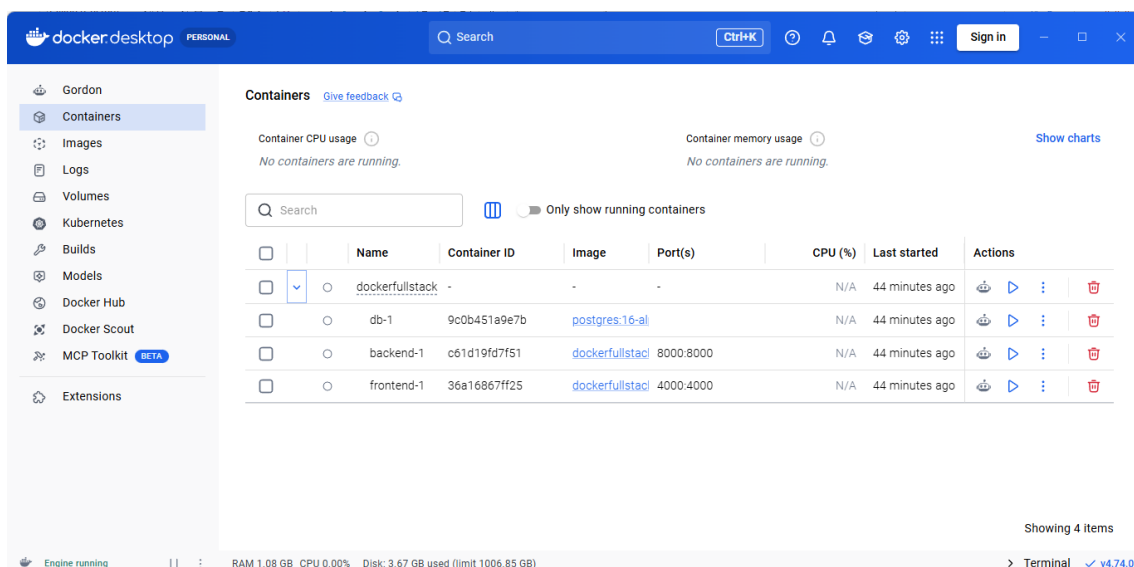
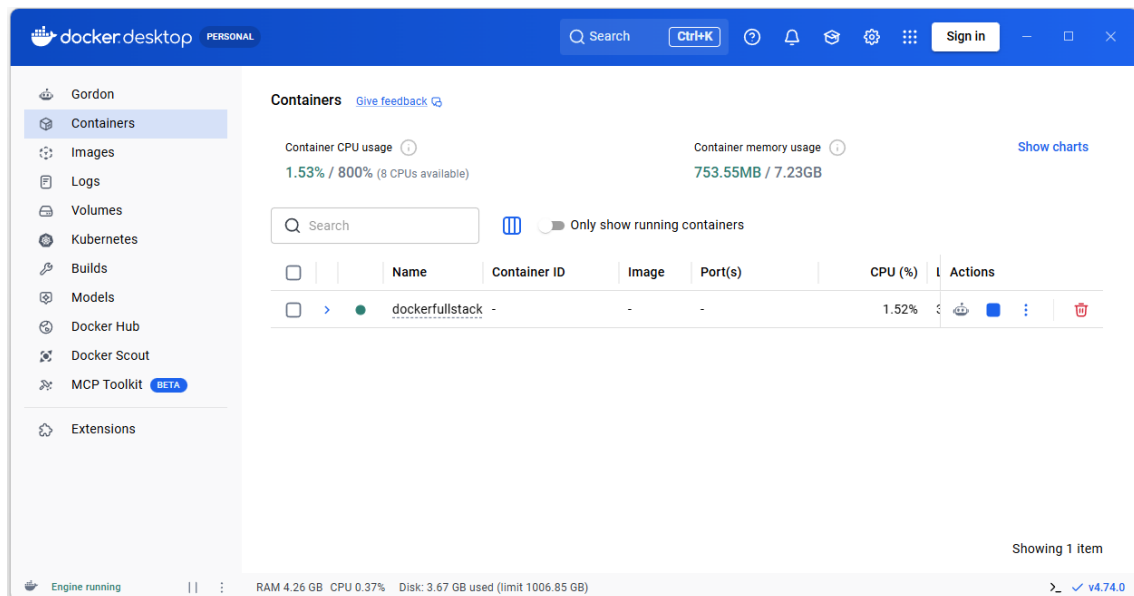
También se utilizó Docker Desktop para comprobar gráficamente que los servicios estaban ejecutándose correctamente.

Paso 12 – Resultado Final

Finalmente se logró levantar correctamente:

- PostgreSQL,
- Backend Django,
- Frontend Angular.

Todos los servicios quedaron integrados dentro de Docker Compose utilizando contenedores independientes.



11. Conclusiones

El uso de Docker permitió resolver el problema relacionado con diferencias entre entornos de desarrollo.

La contenerización facilitó considerablemente la ejecución y despliegue de la aplicación completa.

Docker Compose permitió administrar múltiples servicios de manera sencilla y organizada.

La utilización de multi-stage builds ayudó a optimizar las imágenes y reducir archivos innecesarios.

Durante el desarrollo se comprendió la importancia de:

- automatización,
- portabilidad,

- seguridad,
- y buenas prácticas DevOps.

Finalmente, esta actividad permitió integrar todos los conocimientos adquiridos durante el semestre dentro de un entorno full stack completamente funcional.

12. Recomendaciones

- Implementar HTTPS en producción.
- Utilizar imágenes aún más optimizadas.
- Implementar monitoreo de contenedores.
- Configurar backups automáticos para PostgreSQL.

13. Referencias Bibliográficas

Docker Inc. (2025). *Docker documentation*. <https://docs.docker.com/>

Docker Inc. (2025). *Docker Compose documentation*. <https://docs.docker.com/compose/>

Django Software Foundation. (2025). *Django documentation*. <https://docs.djangoproject.com/>

Google. (2025). *Angular documentation*. <https://angular.dev/>

PostgreSQL Global Development Group. (2025). *PostgreSQL documentation*.
<https://www.postgresql.org/docs/>

<https://chatgpt.com>

Repositorios

Backend Django

<https://github.com/Daniel-Metal/cemlad-teaching-django>

Frontend Angular

<https://github.com/Daniel-Metal/cemlad-teaching-angular>